

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования
Санкт-Петербургский национальный исследовательский университет ИТМО
Мегафакультет трансляционных информационных технологий

Факультет информационных технологий и программирования

Лабораторная работа №3

По дисциплине «Проектирование и реализация баз данных»
Вариант №19

Выполнил студент группы №К3240,
Гаенко Анастасия Константиновна

Преподаватель:
Говорова Марина Михайловна



Санкт-Петербург
2026

I

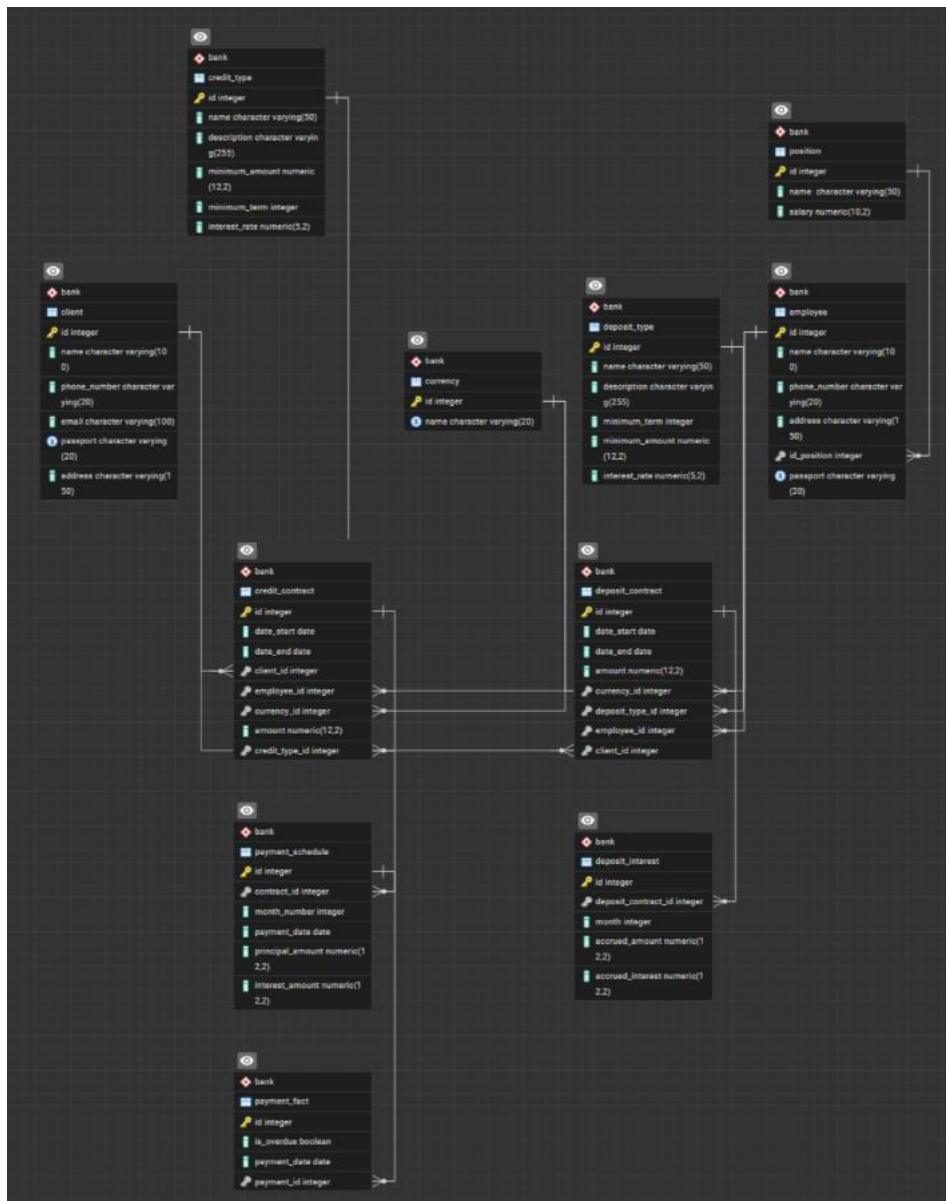
Цель работы: овладеть практическими навыками создания запросов на выборку и модификацию данных в СУБД PostgreSQL, разработки представлений, использования подзапросов, а также анализа производительности запросов с применением индексов и команды EXPLAIN.

II

Практическое задание:

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL.
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.
3. Изучить графическое представление запросов и историю запросов в Query Tool.
4. Создать простой и составной индексы для выбранных запросов.
5. Выполнить анализ запросов без индексов и с индексами с использованием команды EXPLAIN.
6. Сравнить время выполнения запросов и сделать выводы.

III



VI

1. SELECT, VIEW

1) Найти вкладчика, имеющего на текущий день несколько вкладов.

Query

Query History

```
1 select name
2 from bank.client
3 where id in (select count(*)
4 from bank.deposit_contract
5 group by client_id
6 having client_id > 1)
```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 1

✎

🖨️

 Page No: 1

	name character varying (100) 🔒
1	Иванов Иван Иванович

- 2) Найти вкладчика, имеющего вклады в нескольких видах валюты на текущий день.

The screenshot shows a database query editor with a dark theme. The top section is titled "Query" and contains an SQL query. Below the query is a "Data Output" section showing the results of the query in a table format. The query is as follows:

```
1 SELECT
2     c.id,
3     c.name,
4     COUNT(DISTINCT dc.currency_id) AS currency_count
5 FROM bank.client c
6 JOIN bank.deposit_contract dc ON c.id = dc.client_id
7 GROUP BY c.id, c.name
8 HAVING COUNT(DISTINCT dc.currency_id) > 1;
```

The "Data Output" section shows the results of the query. It includes a toolbar with icons for adding, deleting, and refreshing data, as well as a "SQL" button. The results are displayed in a table with the following columns: id, name, and currency_count. The table shows one row of data for a client named "Петров Петр Петрович" with a currency_count of 2.

	id [PK] integer	name character varying (100)	currency_count bigint
1	2	Петров Петр Петрович	2

3) Вывести данные вкладчика, имеющего максимальный вклад в английских фунтах.

Query	Query History
1	SELECT
2	c.*,
3	dc.amount,
4	cur.name AS currency
5	FROM bank.client c
6	JOIN bank.deposit_contract dc ON c.id = dc.client_id
7	JOIN bank.currency cur ON dc.currency_id = cur.id
8	WHERE cur.name = 'GBP'
9	AND dc.amount = (
10	SELECT MAX(dc2.amount)
11	FROM bank.deposit_contract dc2
12	JOIN bank.currency cur2 ON dc2.currency_id = cur2.id
13	WHERE cur2.name = 'GBP'
14	);

Data Output	Messages	Notifications
Showing rows: 1 to 1 of 1		
id integer	name character varying (100)	phone_number character varying (20)
1	Петров Петр Петрович	+79990000002
2	petrov2@mail.ru	4011 123457
3	Санкт-Петербург, Литейный пр., ...	3000000.00
4	GBP	

4) Какой из вкладов пользовался наибольшей популярностью за истекший календарный год.

Query	Query History
1	WITH deposit_counts AS (
2	SELECT
3	dt.id,
4	dt.name AS deposit_type,
5	COUNT(dc.id) AS contracts_count
6	FROM bank.deposit_type dt
7	JOIN bank.deposit_contract dc ON dt.id = dc.deposit_type_id
8	WHERE EXTRACT(YEAR FROM dc.date_start) = EXTRACT(YEAR FROM CURRENT_DATE) - 1
9	GROUP BY dt.id, dt.name
10	)
11	SELECT *
12	FROM deposit_counts
13	WHERE contracts_count = (
14	SELECT MAX(contracts_count)
15	FROM deposit_counts
16	);

Data Output	Messages	Notifications
Showing rows: 1 to 1 of 1		
id [PK] integer	deposit_type character varying (50)	contracts_count bigint
1	Накопительный	3

5) Кто из сотрудников заключил максимальное число договоров по кредитам за последний месяц.

QueryQuery History

1

WITH employee_counts AS (
2SELECT
3e.id,
4e.name AS employee_name,
5COUNT(cc.id) AS credit_contract_count
6FROM bank.employee e
7JOIN bank.credit_contract cc ON e.id = cc.employee_id
8WHERE cc.date_start >= CURRENT_DATE - INTERVAL '1 month'
9GROUP BY e.id, e.name
10)
11SELECT *
12FROM employee_counts
13WHERE credit_contract_count = (
14SELECT MAX(credit_contract_count)
15FROM employee_counts
16);

Data OutputMessagesNotifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	id [PK] integer	employee_name character varying (100)	credit_contract_count bigint
1	2	Федоров Максим Андреев...	1

6) Вывести список вкладчиков, у которых срок вклада истекает завтра и суммы начислений, которые могут быть ими востребованы.

Query

Query History

```

1  SELECT
2      c.id,
3      c.name AS client_name,
4      dc.id AS deposit_contract_id,
5      dc.date_end,
6      dc.amount AS deposit_amount,
7      COALESCE(SUM(di.accrued_amount), 0) AS total_accrued_amount,
8      dc.amount + COALESCE(SUM(di.accrued_amount), 0) AS total_available_amount
9  FROM bank.client c
10 JOIN bank.deposit_contract dc ON c.id = dc.client_id
11 LEFT JOIN bank.deposit_interest di ON dc.id = di.deposit_contract_id
12 WHERE dc.date_end = CURRENT_DATE + INTERVAL '1 day'
13 GROUP BY c.id, c.name, dc.id, dc.date_end, dc.amount;

```

Data Output

Messages

Notifications

+

📄

▼

🗑️

📄

📄

📄

📄

SQL

Showing rows: 1 to 1

	id integer	client_name character varying (100)	deposit_contract_id integer	date_end date	deposit_amount numeric (12,2)	total_accrued_amount numeric	total_available_amount numeric
1	1	Иванов Иван Иванович	11	2026-04-28	3000000.00	100000.00	3100000.00

7) Вывести список сотрудников, заключивших договоры по вкладам на максимальную сумму за последний месяц.

Query

Query History

```

1  SELECT
2      e.id,
3      e.name AS employee_name,
4      SUM(dc.amount) AS total_deposit_amount
5  FROM bank.employee e
6  JOIN bank.deposit_contract dc ON e.id = dc.employee_id
7  WHERE dc.date_start >= CURRENT_DATE - INTERVAL '1 month'
8  GROUP BY e.id, e.name
9  HAVING SUM(dc.amount) = (
10     SELECT MAX(total_amount)
11     FROM (
12         SELECT SUM(dc2.amount) AS total_amount
13         FROM bank.deposit_contract dc2
14         WHERE dc2.date_start >= CURRENT_DATE - INTERVAL '1 month'
15         GROUP BY dc2.employee_id
16     ) AS sums
17 );

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗑️

📥

⬇️

📈

SQL

	id [PK] integer	employee_name character varying (100)	total_deposit_amount numeric
1	1	Орлова Наталья Игоревна	3220000.00

- 1) содержащее сведения обо всех сотрудниках банка и заключенных ими договорах по кредитам за прошедший месяц

```

CREATE OR REPLACE VIEW bank.employee_credit_contracts_last_month AS
SELECT
    e.id AS employee_id,
    e.name AS employee_name,
    e.phone_number,
    e.id_position,
    cc.id AS credit_contract_id,
    cc.date_start,
    cc.date_end,
    cc.amount,
    c.name AS client_name,
    ct.name AS credit_type
FROM bank.employee e
JOIN bank.credit_contract cc ON e.id = cc.employee_id
JOIN bank.client c ON cc.client_id = c.id
JOIN bank.credit_type ct ON cc.credit_type_id = ct.id

```

```
WHERE cc.date_start >= CURRENT_DATE - INTERVAL '1 month';
```

Query Query History

```
1 SELECT *
2 FROM bank.employee_credit_contracts_last_month;
```

Data Output Messages Notifications

Showing rows: 1 to 1 of 1

employee_id	employee_name	phone_number	id_position	credit_contract_id	date_start	date_end	amount	client_name	credit_type
integer	character varying (100)	character varying (20)	integer	integer	date	date	numeric (12,2)	character varying (100)	character varying (50)
2	Федоров Максим Андреевич	+790000000002	2	1	2026-04-01	2026-10-01	100000.00	Петров Петр Петрович	Потребительский

2) найти клиентов банка, имеющих задолженности по кредитам

```
CREATE OR REPLACE VIEW bank.clients_with_credit_debt AS
SELECT
    c.id AS client_id,
    c.name AS client_name,
    cc.id AS credit_contract_id,
    cc.amount AS credit_amount,
    COALESCE(SUM(ps.principal_amount + ps.interest_amount), 0) AS planned_sum,
    COALESCE(SUM(
        CASE
            WHEN pf.payment_date IS NOT NULL
            THEN ps.principal_amount + ps.interest_amount
            ELSE 0
        END
    ), 0) AS paid_sum,
    COALESCE(SUM(ps.principal_amount + ps.interest_amount), 0)
    -
    COALESCE(SUM(
        CASE
            WHEN pf.payment_date IS NOT NULL
            THEN ps.principal_amount + ps.interest_amount
            ELSE 0
        END
    ), 0) AS debt_sum
FROM bank.client c
JOIN bank.credit_contract cc ON c.id = cc.client_id
JOIN bank.payment_schedule ps ON cc.id = ps.contract_id
LEFT JOIN bank.payment_fact pf ON ps.id = pf.payment_id
GROUP BY c.id, c.name, cc.id, cc.amount
HAVING
    COALESCE(SUM(ps.principal_amount + ps.interest_amount), 0)
```

```

>
COALESCE(SUM(
    CASE
        WHEN pf.payment_date IS NOT NULL
        THEN ps.principal_amount + ps.interest_amount
        ELSE 0
    END
), 0);

```

Query Query History

```

1 Select *
2 from bank.clients_with_credit_debt

```

Data Output Messages Notifications

	client_id integer	client_name character varying (100)	credit_contract_id integer	credit_amount numeric (12,2)	planned_sum numeric	paid_sum numeric	debt_sum numeric
1	2	Петров Петр Петрович	1	100000.00	34700.00	17500.00	17200.00
2	5	Смирнова Мария Олегов...	3	2600000.00	69800.00	0	69800.00

2. INSERT, UPDATE, DELETE

fpi_bd/postgres@... X Untitled* X Processes X Untitled* X fpi_bd_test/postgr... X Untitled* X fpi_bd

fpi_bd_test/postgres@PostgreSQL 18

Query Query History

```
1 SELECT *
2 FROM bank.deposit_contract
3 ORDER BY id;
```

Data Output Messages Notifications

Showing rows: 1 to 4

	id [PK] integer	date_start date	date_end date	amount numeric (12,2)	currency_id integer	deposit_type_id integer	employee_id integer	client_id integer
1	1	2026-04-01	2026-10-01	120000.00	1	1	1	1
2	2	2026-03-15	2027-03-15	300000.00	1	2	1	3
3	3	2026-04-05	2026-07-05	15000.00	1	3	3	5
4	4	2026-05-01	2026-11-01	200000.00	1	1	1	1

```
INSERT INTO bank.deposit_contract (
    id,
    date_start,
    date_end,
    amount,
    currency_id,
    deposit_type_id,
    employee_id,
    client_id
)
VALUES (
    5,
    '2026-05-06',
    '2027-11-01',
    2900000,
    (SELECT id FROM bank.currency WHERE name = 'RUB'),
    (SELECT id FROM bank.deposit_type WHERE name = 'Накопительный'),
    (SELECT id FROM bank.employee WHERE name = 'Орлова Наталья Игоревна'),
    (SELECT id FROM bank.client WHERE name = 'Иванов Иван Иванович')
)
```

);

Query

Query History

1

2

3

SELECT *

FROM bank.deposit_contract

ORDER BY id;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

📦

⬇️

📈

SQL

Showing rows: 1 to 5

✎

🔍

	id [PK] integer	date_start date	date_end date	amount numeric (12,2)	currency_id integer	deposit_type_id integer	employee_id integer	client_id integer
1	1	2026-04-01	2026-10-01	120000.00	1	1	1	1
2	2	2026-03-15	2027-03-15	300000.00	1	2	1	3
3	3	2026-04-05	2026-07-05	15000.00	1	3	3	5
4	4	2026-05-01	2026-11-01	200000.00	1	1	1	1
5	5	2026-05-06	2027-11-01	2900000.00	1	1	1	1

fpi_bd/postgres@... x Untitled* x Processes x Untitled* x fpi_bd_test/postgr... x Untitled* x fp

fpi_bd_test/postgres@PostgreSQL 18

Query Query History

```
1 SELECT *
2 FROM bank.credit_contract
3 WHERE client_id = (
4     SELECT id
5     FROM bank.client
6     WHERE name = 'Смирнова Мария Олеговна'
7 );
```

Data Output Messages Notifications

Showing rows: 1 to 1

	id [PK] integer	date_start date	date_end date	client_id integer	employee_id integer	currency_id integer	amount numeric (12,2)	credit_type_id integer
1	3	2026-01-10	2031-01-10	5	2	1	2550000.00	3

```
UPDATE bank.credit_contract
SET amount = amount + 50000
WHERE client_id = (
    SELECT id
    FROM bank.client
    WHERE name = 'Смирнова Мария Олеговна'
);
```

Query History

```
SELECT *
FROM bank.credit_contract
WHERE client_id = (
  SELECT id
  FROM bank.client
  WHERE name = 'Смирнова Мария Олеговна'
);
```

Data Output Messages Notifications

Showing rows: 1 to 1								
id		date_start	date_end	client_id	employee_id	currency_id	amount	credit_type_id
[PK] integer		date	date	integer	integer	integer	numeric (12,2)	integer
	3	2026-01-10	2031-01-10	5	2	1	2600000.00	3

Query

Query History

1

2

3

SELECT *

FROM bank.payment_fact

WHERE is_overdue = true;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	id [PK] integer	is_overdue boolean	payment_date date	payment_id integer
1	2	true	[null]	2

```
DELETE FROM bank.payment_fact
WHERE payment_id IN (
  SELECT id
  FROM bank.payment_schedule
  WHERE id IN (
    SELECT payment_id
    FROM bank.payment_fact
    WHERE is_overdue = true
  )
);
```

Query

Query History

1

2

3

SELECT *

FROM bank.payment_fact

WHERE is_overdue = true;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

id	is_overdue	payment_date	payment_id
[PK] integer	boolean	date	integer

3. INDEX

Query

Query History

1

EXPLAIN ANALYZE

2

SELECT *

3

FROM bank.client

4

WHERE name = 'Иванов Иван Иванович';

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing

QUERY PLAN

text

🔒

1

Seq Scan on client (cost=0.00..1.06 rows=1 width=874) (actual time=0.084..0.085 rows=1.00 loop...

2

Filter: ((name)::text = 'Иванов Иван Иванович'::text)

3

Rows Removed by Filter: 4

4

Buffers: shared hit=1

5

Planning Time: 0.168 ms

6

Execution Time: 0.106 ms

```
CREATE INDEX idx_client_name
ON bank.client(name);
```

No limit

Query

Query History

1

EXPLAIN ANALYZE

2

SELECT *

3

FROM bank.client

4

WHERE name = 'Иванов Иван Иванович';

Data Output

Messages

Notifications

SQL

Sho

QUERY PLAN

text

1

Seq Scan on client (cost=0.00..1.06 rows=1 width=874) (actual time=0.013..0.014 rows=1.00 loop...

2

Filter: ((name)::text = 'Иванов Иван Иванович')::text

3

Rows Removed by Filter: 4

4

Buffers: shared hit=1

5

Planning:

6

Buffers: shared hit=16 read=1

7

Planning Time: 0.959 ms

8

Execution Time: 0.028 ms

Query

Query History

1

EXPLAIN ANALYZE

2

SELECT *

3

FROM bank.credit_contract

4

WHERE client_id = 5

5

AND date_start >= '2026-01-01';

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing row

QUERY PLAN

text

🔒

1

Seq Scan on credit_contract (cost=0.00..1.04 rows=1 width=44) (actual time=0.020..0.020 rows=1.00 loop...

2

Filter: ((date_start >= '2026-01-01'::date) AND (client_id = 5))

3

Rows Removed by Filter: 2

4

Buffers: shared hit=1

5

Planning Time: 0.088 ms

6

Execution Time: 0.035 ms

```
CREATE INDEX idx_credit_contract_client_date
ON bank.credit_contract(client_id, date_start);
```

The screenshot shows a PostgreSQL query editor interface. The top toolbar includes icons for file operations, query execution, and settings. The main area displays a SQL query:

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM bank.credit_contract
4 WHERE client_id = 5
5    AND date_start >= '2026-01-01';
```

Below the query editor, the 'Data Output' tab is active, showing the 'QUERY PLAN' for the executed query. The plan details are as follows:

Step	Operation
1	Seq Scan on credit_contract (cost=0.00..1.04 rows=1 width=44) (actual time=0.011..0.011 rows=1.00 loop...)
2	Filter: ((date_start >= '2026-01-01'::date) AND (client_id = 5))
3	Rows Removed by Filter: 2
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=22 read=1
7	Planning Time: 0.802 ms
8	Execution Time: 0.024 ms

```
DROP INDEX bank.idx_client_name;
DROP INDEX bank.idx_credit_contract_client_date;
```

V

Выводы: в ходе выполнения лабораторной работы были получены практические навыки создания запросов на выборку данных в PostgreSQL, разработки представлений, выполнения запросов на модификацию данных с использованием подзапросов, а также работы с индексами. Были составлены SELECT-запросы к базе данных предметной области «Банк», созданы представления для удобного просмотра связанных данных,

выполнены запросы INSERT, UPDATE и DELETE с подзапросами. Также были созданы простой и составной индексы, выполнен анализ планов запросов с помощью EXPLAIN ANALYZE и проведено сравнение времени выполнения запросов до и после создания индексов. В результате были закреплены навыки работы с Query Tool, представлениями, подзапросами и индексами в pgAdmin 4.